



Project Report

ΑΚΑΔΗΜΑΪΚΟ ΕΤΟΣ 2025 – 2026

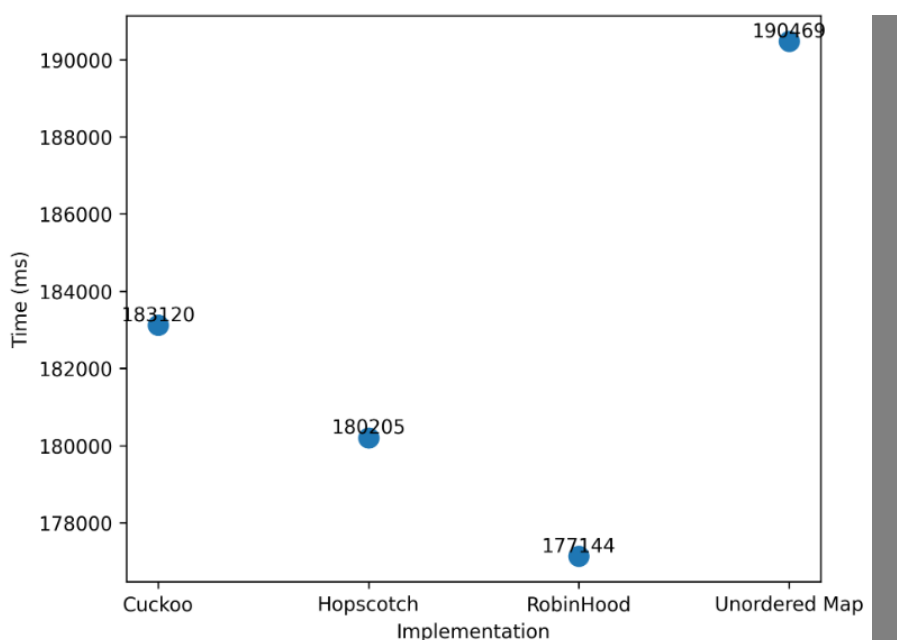
Επώνυμο – Όνομα	Αριθμός Μητρώου	Email
Μπουχαλάκης Ανδρέας	1115202200113	sdi2200113@di.uoa.gr
Τζάκας Δημήτριος	1115202200182	sdi2200182@di.uoa.gr

Εργασία 1:

Για το πρώτο παραδοτέο μας ζητήθηκε να υλοποιήσουμε 3 διαφορετικά Hash Table και να συγκρίνουμε την απόδοσή τους σε σχέση με το unordered map της stl. Τα τρία αυτά hash table που ζητήθηκαν ήταν:

- **Hopscotch Hashtable:** Είναι ένας πίνακας κατακερματισμού που η βασική λειτουργία του είναι ότι κάνει hash τα στοιχεία μόνο εντός μία γειτονιάς, δηλαδή αν ένα στοιχείο hashαρει στη θέση i και είναι ήδη δεσμευμένη από στοιχείο που αντιστοιχεί στη θέση αυτή, το στοιχείο μας μπορεί να πάει το πολύ μέχρι τη θέση $i + n$ όπου n το μέγεθος της γειτονιάς, αν η γειτονιά είναι γεμάτη προσπαθούμε να μετακινήσουμε στοιχεία αλλιώς κάνουμε rehash τον πίνακα.
- **RobinHood HashTable:** Είναι ένας πίνακας κατακερματισμού που η βασική του λειτουργία είναι κάθε στοιχείο να απέχει το λιγότερο δυνατό από τη θέση την οποία αντιστοιχεί. Αν το στοιχείο j αντιστοιχεί στη θέση i και το στοιχείο k στη θέση n όπου $n > i$ αν το j δεν μπορεί να μπει στη θέση που του αναλογεί και φτάσει σε μία θέση x ($x > n > i$) και εκεί υπάρχει το στοιχείο k , το j θα μπει αντί για το k και αυτό με τη σειρά του θα ψάξει για επόμενη θέση.
- **Cuckoo HashTable:** Αποτελείται από 2 πίνακες κατακερματισμού οι οποίοι έχουν 2 διαφορετικές hash function, αν 2 στοιχεία συμπέσουν σε μία θέση, το στοιχείο που είναι να εισαχθεί διώχνει το ήδη υπάρχον στοιχείο το οποίο με τη σειρά του ψάχνει να βρει θέση στον επόμενο πίνακα.

Επίσης βάλαμε τα hash table να δέχονται σαν όρισμα το πόσα στοιχεία περιμένουν και τα χτίζουμε με μέγεθος διπλάσιο αυτού του αριθμού έτσι ώστε να μην χρειαστεί rehashing. Ακολουθούν διαγράμματα με τα αποτελέσματά μας και ο σχολιασμός τους.



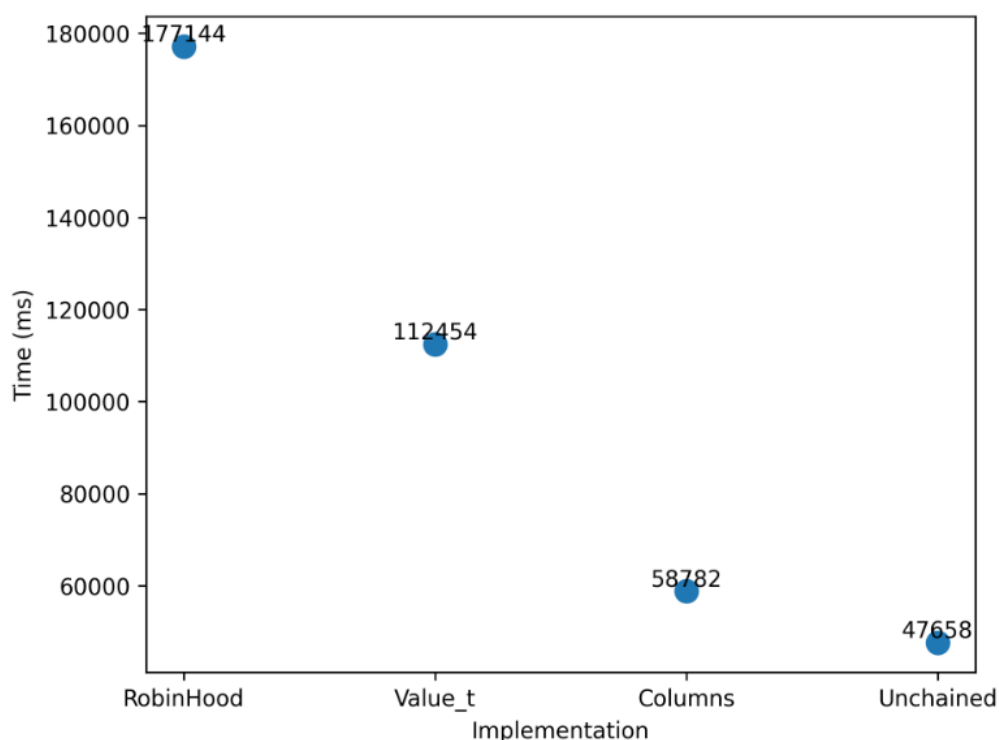
Από το παραπάνω διάγραμμα παρατηρούμε ότι όλες οι υλοποιήσεις ήταν αρκετά πιο αποδοτικές από την υλοποίηση του unordered map δηλαδή την αρχική. Μάλιστα στην περίπτωση του Robin Hood παρατηρούμε παραπάνω από 10 δευτερόλεπτα βελτίωση στον χρόνο. Οι βελτιώσεις που είδαμε είναι λογικές καθώς μιλάμε για πολύ πιο αποδοτικούς πίνακες κατακερματισμού.

Εργασία 2: Στο δεύτερο παραδοτέο, αρχικά μας ζητήθηκε να περιορίσουμε τις αντιγραφές των string στα ενδιάμεσα join. Αυτό το πετύχαμε χρησιμοποιώντας μία νέα δομή τη value_t η οποία μπορεί να είναι είτε sting είναι integer είναι NULL, τα value_t πλέον δείχνουν ένα από τα παραπάνω και ο join algorithm αλλάζει και γίνεται πλέον μόνο μεταξύ integers. Τέλος αφού γίνουν τα απαραίτητα join, κάνει materialize τα αποτελέσματα δηλαδή τα μετατρέπει από value_t σε αυτό που τους αντιστοιχεί (string, integer, NULL).

Στη συνέχεια, ζητήθηκε να κάνουμε το πρόγραμμα περισσότερο Cache friendly, δηλαδή πλέον τα δεδομένα δεν θα αποθηκεύονται σε γραμμές αλλά σε στήλες, με αυτόν τον τρόπο θα ελαχιστοποιήσουμε τα Cache misses. Για την παραπάνω υλοποίηση χρησιμοποιήσαμε μία δομή Paged Column δομή η οποία κρατάει τα αποτελέσματα σε σελίδες που είναι βασισμένα σε στήλες, το μέγεθος που επιλέχθηκε για τη σελίδα έπειτα από πειράματα είναι 4096, τα πειράματα που έγιναν ήταν (512-8192) με το 4096 να παρουσιάζει την καλύτερη δυνατή συμπεριφορά. Στη συνέχεια αλλάξαμε την execute ώστε να δουλεύει με στήλες και όχι με γραμμές όπως έκανε ως τώρα, αλλά και την execute scan η οποία η οποία πλέον δεν επιστρέφει γραμμές αλλά γράφει απευθείας στο paged column, τέλος το hash join πλέον κάνει επαναλήψεις γύρω από στήλες και όχι γραμμές.

Στο τελευταίο ερώτημα του παραδοτέου μας ζητήθηκε να υλοποιήσουμε ένα καινούριο hashtable το Unchained Hashtable, το οποίο θα αποτελέσει βασικό παράγοντα για το 3^ο παραδοτέο. Για την υλοποίηση του συμβουλευτήκαμε το paper για το μέγεθος που θα έχει καθώς και την hash function την οποία χρησιμοποιεί η οποία είναι η crc_32, έπειτα και από συγκρίσεις με άλλες hash function όπως η Fibonacci. Το build του hash table γίνεται ως εξής, αρχικά συλλέγει tuples με τις τιμές που αναμένεται να εισαχθούν, αφού μαζέψει όλες τις τιμές κάνει το build. Χρησιμοποιούμε bloom filters όπως ζητείται για να δείξουμε την πιθανότητα ύπαρξης του στοιχείου στον πίνακα. Η find γυρίζει ένα vector από κάθε στοιχείο που υπάρχει σε ένα κελί, καθώς διπλότυπα στοιχεία αποθηκεύονται στο ίδιο.

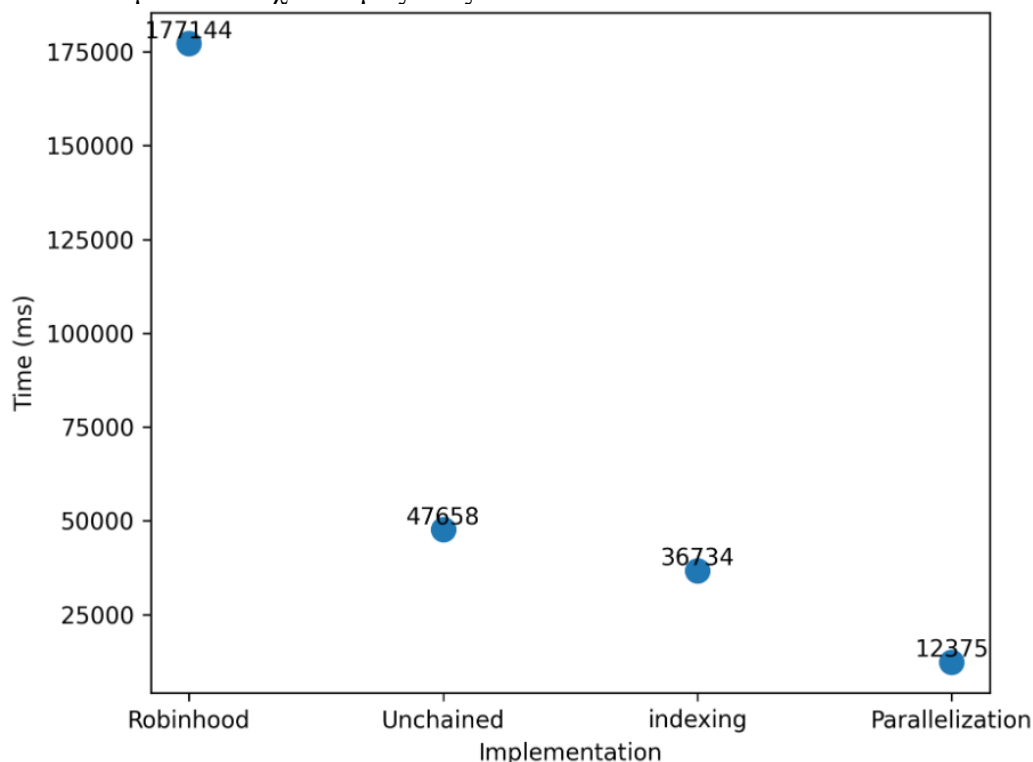
Ακολουθούν διαγράμματα με τα αποτελέσματα μας και ο σχολιασμός τους:



Αρχικά με την πρώτη βελτιστοποίηση, δηλαδή με τη χρήση `value_t` παρατηρούμε πως ο χρόνος ήδη μειώνεται σημαντικά, πιο συγκεκριμένα παρατηρούμε βελτίωση κοντά στο 40% από την υλοποίηση του Robinhood, αυτό είναι λογικό καθώς ο χρόνος ο οποίος σπαταλούταν για την αντιγραφή των strings στα ενδιάμεσα αποτελέσματα ήταν αρκετός ειδικά όταν μιλάμε για μεγάλα και πολλά string. Πλέον το `join` γίνεται μόνο μεταξύ `integer` και είναι πολύ πιο απλό και πολύ πιο γρήγορο. Στη συνέχεια παρατηρούμε ότι με τον νέο τρόπο αποθήκευσης δεδομένων (`column store`) ο χρόνος μειώνεται ακόμη περισσότερο και η επιτάχυνση που παρατηρούμε από το Robinhood είναι της τάξης του 67%. Το πρόγραμμα πλέον είναι `cache friendly` πράγμα που σημαίνει ότι έχει πολύ λιγότερες αστοχίες στην `cache` και πληρώνει ολοένα και λιγότερους κύκλους στη μεταφορά δεδομένων σε αυτήν, οπότε η βελτίωση που παρατηρούμε είναι η αναμενόμενη. Τέλος και με την τελευταία προσθήκη, δηλαδή το `unchained hash table` παρατηρούμε ότι ο χρόνος έχει μειωθεί ακόμα περισσότερο από πριν, πράγμα που περιμέναμε καθώς το νέο `hashtable` είναι πολύ πιο αποδοτικό από τα προηγούμενα.

Εργασία 3: Τέλος για το τελευταίο παραδοτέο της εργασίας μας ζητήθηκε αρχικά να τροποποιήσουμε το `indexing`, με αυτόν το τρόπο το αντιγράφονται μόνο τα `strings` ενώ για οι `integers` αντιγράφονται όπως είναι στην σελίδα και δείχνονται μέσω `index`, με αυτό τον τρόπο χρειάζεται να κάνουμε `materialize` μόνο τα `strings` καθώς τα `integer` τα έχουμε έτοιμα. Στη συνέχεια μας ζητήθηκε να παραλληλοποιήσουμε το `join algorithm`, για την οποία παραλληλοποίηση χρησιμοποιήσαμε `OpenMP`. Αρχικά, φτιάχνουμε έναν `bump allocator` και συλλέγουμε παράλληλα όλα τα `tuples` που πρόκειται να εισαχθούν στο `hashtable`, για την κατασκευή του `hashtable` παίρνουμε τα `tuples` που έχουν περαστεί και αρχίσαμε και δημιουργούμε το `hashtable` γράφοντας κάθε `tuple` σε ένα προσωρινές δομές καθώς και μετρώντας τα έτσι ώστε να ξέρουν οι ζώες αποθήκευσης πόσα `tuples` περιμένουν. Στην συνέχεια κάνουμε ένα `prefix sum` των αποτελεσμάτων και στη συνέχεια αποθηκεύουμε παράλληλα τα `tuples` στα `partitions` που αντιστοιχούν. Τέλος αντιγράφονται τα `tuples` στην κανονική τους θέση και φτιάχνουμε `directory` να έχει σωστό `offset`. Επίσης παραλληλοποιήσαμε και το `probing phase` έτσι ώστε τα κάθε `thread` να διαβάζει τοπικά τα αποτελέσματα από τη `find` του `hashtable` ενώ στη συνέχεια το `aggregation` γίνεται από ένα και μόνο `thread`. Τέλος ζητήθηκε να εφαρμόσουμε `work stealing` το οποίο εφαρμόζεται, αυτόματα από την `OpenMP` καθώς όποιο `thread` τελειώνει τη δουλειά του μπορεί να πάρει την επόμενη δουλειά. Έτσι κάποια `threads` με ελαφρή έργο δεν θα μείνουν ποτέ ανεκμετάλλευτα.

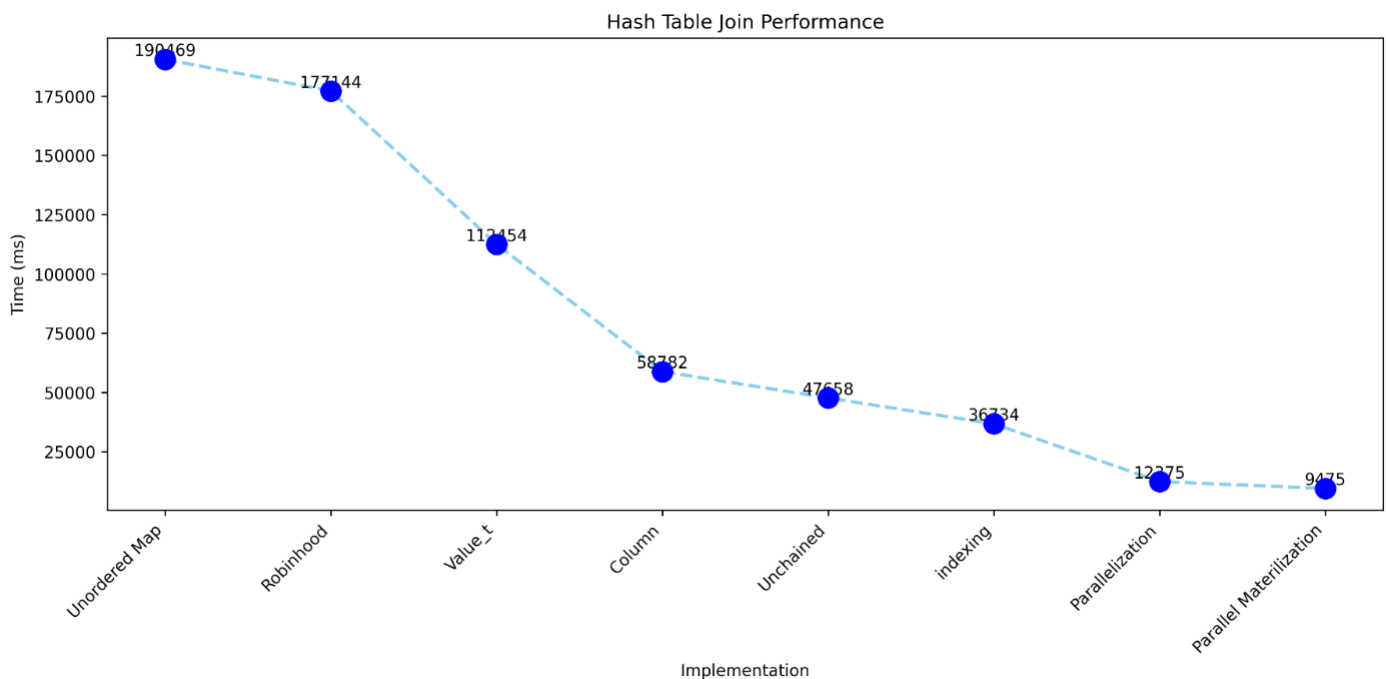
Ακολουθούν αποτελέσματα και σχολιασμός τους:



Πλέον οι χρόνοι που παρατηρούμε είναι σημαντικά βελτιωμένοι, στην περίπτωση του indexing σε σχέση με το robinhood παρατηρούμε 79% επιτάχυνση, πράγμα που είναι λογικό καθώς πλέον έχουμε materialize μόνο στα string ενώ στους integer έχουμε απλά indexes. Ενώ στην συνέχεια με την προσθήκη παραλληλίας στο hashtable καθώς και στο join algorithm παρατηρούμε ότι πλέον έχουμε επιτάχυνση 93%, αναμενόμενη συμπεριφορά αφού πλέον πολλές από τις βαριές δουλειές τρέχουν παράλληλα και έτσι ο χρόνος μειώνεται ανάλογα με το πλήθος των thread που χρησιμοποιούμε.

Για το 3^ο παραδοτέο το Chunk size που επιλέξαμε έπειτα από πειράματα είναι 4096 ενώ στον bump allocator το chunk size αντιστοιχεί σε 16 kb. Τα παραπάνω αποτελέσματα είναι επιλεγμένα πειραματικά. Επίσης ο αριθμός των thread που χρησιμοποιούμε είναι ο μέγιστος που μπορεί να προσφέρει το μηχάνημα στο probin fase ενώ στο tuple collection είναι ανάλλογος του πλήθους των tuple που έχουμε να συλλέξουμε. Τέλος, επιλέξαμε για την υλοποίηση της παραλληλίας να επιλέξουμε την OpenMP καθώς είναι απλή στη χρήση της και πολύ καλή επιλογή για παραλληλία.

Δική μας βελτίωση: Στα πλαίσια της βελτίωσης του αλγορίθμου σκεφτήκαμε και μία δική μας προσθήκη που απέδωσε αρκετά σε χρόνο, η ιδέα είναι να συλλέγουμε τα αποτελέσματα παράλληλα στην execute, έτσι μπορούν να γίνουν παράλληλα materialize τα οποία είναι σχετικά ακριβά σε χρόνο. Για την υλοποίηση αυτή χρησιμοποιήσαμε επίσης OpenMP ακολουθεί διάγραμμα με τους τελικούς χρόνους.



Τελικά η επιτάχυνση που πετύχαμε είναι 94.5% τα ποσοστά βελτίωσης είναι όλα σε σχέση με το Robinhood

Τα χαρακτηριστικά του συστήματος που τρέξαμε τα πειράματα είναι:

- Ubuntu 22.04
- Intel Core i7 (10 cores – 16 threads)
- 32 GB RAM